

Project: Advanced Drone Technology.

Project: Introduction to Guidance and Control Using Python Simulations



INDIA SPACE LAB

Project Report

17th June to 31th July 2026

Student Name: Bhosale Pranali Pravin

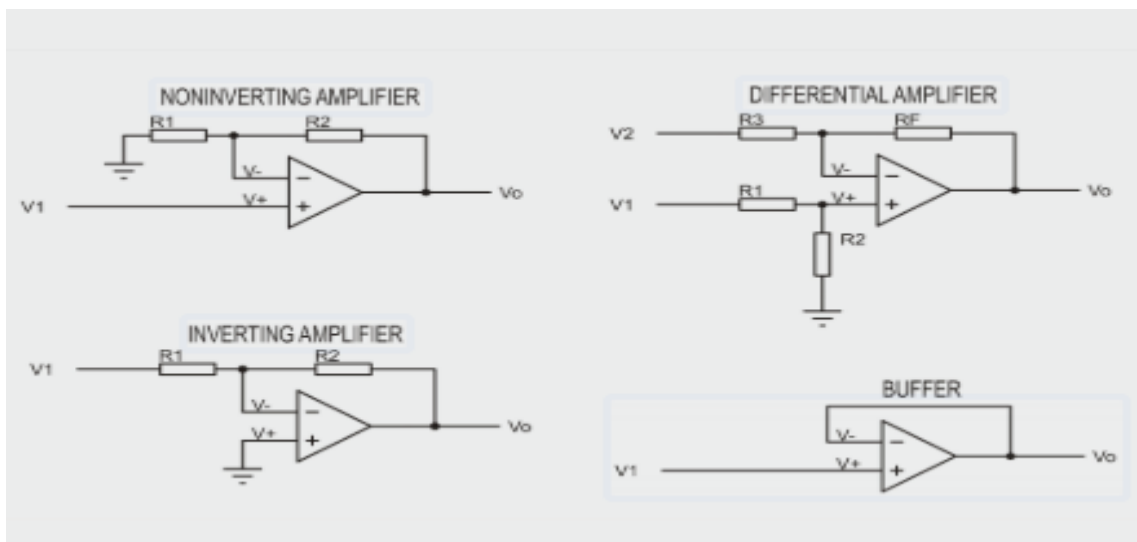
Institute Name: DKTE Society's Textile & Engineering Institute

Institute Roll no.: 24UET320

Enrollment no.: ISL-578965

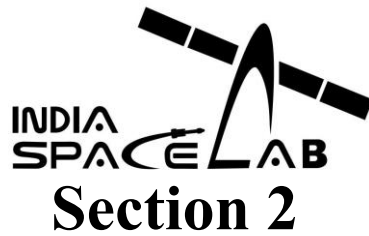
Objectives & Introduction

- Briefly introduce the purpose of the project. Explain the role of feedback loops in autonomous systems and why guidance, navigation, and control (GNC) are vital for modern drones and unmanned marine vehicles.
- Key Concept:** Briefly define what a Proportional-Integral-Derivative (PID) controller does.



A PID controller (Proportional-Integral-Derivative controller) is a feedback control loop mechanism widely used in industrial control systems and autonomous vehicles (like drones) to regulate processes. It continuously calculates an error value—the difference between a desired target setpoint (such as a target altitude) and a measured process variable—and applies a correction based on three combined terms:

- Proportional (P):** Reacts to the current error, applying a corrective force proportional to how far the system is from the target.
- Integral (I):** Accounts for past errors accumulated over time, eliminating any persistent steady-state offset or drift (such as a constant wind pushing a drone down).
- Derivative (D):** Predicts future errors based on the rate of change, acting as a damper to reduce overshoot, dampen oscillations, and stabilize the system's response.



Environment Setup

- State that the implementation was developed using Python 3 within a Google Colab notebook environment, eliminating local hardware dependencies.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output

# Configure matplotlib to output static plot figures cleanly inline
%matplotlib inline
```



Task 1: Drone Altitude PID Controller Tuning

- **System Behavior:** Describe how the drone responds to the introduction of a sudden wind disturbance at $t = 6$ seconds.
- **Tuning Process:** Document the exact Proportional (P), Derivative (D), and Integral (I) gains you settled on using the interactive sliders to stabilize the altitude. Explain *why* you adjusted each gain (e.g., how increasing D reduced overshoots, or how I eliminated steady-state error caused by the wind).
- **Deliverable:** Insert a high-resolution screenshot of your **PID Controller Tuning Result Plot** directly under this section.

Task 1 Simulation Configuration Settings

```
t_max_drone = 12.0
```

```
dt_drone = 0.05
```

```
drone_time_steps = np.arange(0, t_max_drone, dt_drone)
```

```
target_altitude = 10.0 # Target distance marker
```

```
# Global caching arrays to pass runtime telemetry logs to the animation engine
```

```
drone_t_log = []
```

```
drone_z_log = []
```

```
def simulate_drone_flight(Kp, Kd, Ki, wind_setting):
```

```
    global drone_t_log, drone_z_log
```

```
    # Core system state variables
```

```
    altitude = 0.0
```

```
    velocity = 0.0
```

```
    # PID feedback registers
```

```
    accumulated_error = 0.0
```

```
    previous_error = 0.0
```

```
    drone_t_log = []
```

```
    drone_z_log = []
```

```
    for t in drone_time_steps:
```

```
        error = target_altitude - altitude
```



```
accumulated_error += error * dt_drone
derivative_error = (error - previous_error) / dt_drone if t > 0 else 0.0

# Standard PID control signal tracking law
control_thrust = (Kp * error) + (Ki * accumulated_error) + (Kd *
derivative_error)

# Apply environmental wind disturbance step input strictly after 6 seconds
active_wind = wind_setting if t >= 6.0 else 0.0

# State acceleration physics: thrust - gravity + wind force
gravity_constant = 9.81
acceleration = control_thrust - gravity_constant + active_wind

# Euler integration step updates
velocity += acceleration * dt_drone
altitude += velocity * dt_drone

# Hard ground boundary limiter
if altitude < 0.0:
    altitude = 0.0
    velocity = 0.0

drone_t_log.append(t)
drone_z_log.append(altitude)
previous_error = error

def trigger_drone_plot(button_event):
    with drone_ui_output:
        clear_output(wait=True)

        # Execute flight logic processing loop utilizing the slider values
        simulate_drone_flight(w_kp.value, w_kd.value, w_ki.value, w_wind.value)

        # Render static analytical performance evaluation figure
        fig, ax = plt.subplots(figsize=(10, 4))
        ax.plot(drone_t_log, drone_z_log, color='blue', linestyle='-', linewidth=2,
label='Drone Flight Path')
        ax.axhline(target_altitude, color='red', linestyle='--', linewidth=1.5, label='Target
Altitude (10m)')
```



```
ax.axvline(6.0, color='green', linestyle=':', linewidth=1.5, label='Wind Introduced  
(t=6s)') ax.set_title("Drone Altitude PID Step Response & Disturbance Recovery  
Profile")
```

```
ax.set_xlabel("Time Duration (seconds)")  
ax.set_ylabel("Altitude Position (meters)")  
ax.grid(True)  
ax.legend(loc='lower right')  
plt.show()
```

```
# Setup interactive dashboard UI controls
```

```
w_kp = widgets.FloatSlider(value=15.0, min=0.0, max=40.0, step=0.5,  
description='Gain P:')
```

```
w_kd = widgets.FloatSlider(value=5.0, min=0.0, max=20.0, step=0.1, description='Gain  
D:')
```

```
w_ki = widgets.FloatSlider(value=0.1, min=0.0, max=10.0, step=0.1, description='Gain  
I:')
```

```
w_wind = widgets.FloatSlider(value=-6.0, min=-15.0, max=15.0, step=0.5,  
description='Wind Force:')
```

```
btn_run_drone = widgets.Button(description="Run Simulation and Plot",  
button_style='info')
```

```
btn_run_drone.on_click(trigger_drone_plot)
```

```
drone_ui_output = widgets.Output()
```

```
# Pack UI controls vertically
```

```
display(widgets.VBox([w_kp, w_kd, w_ki, w_wind, btn_run_drone, drone_ui_output]))
```

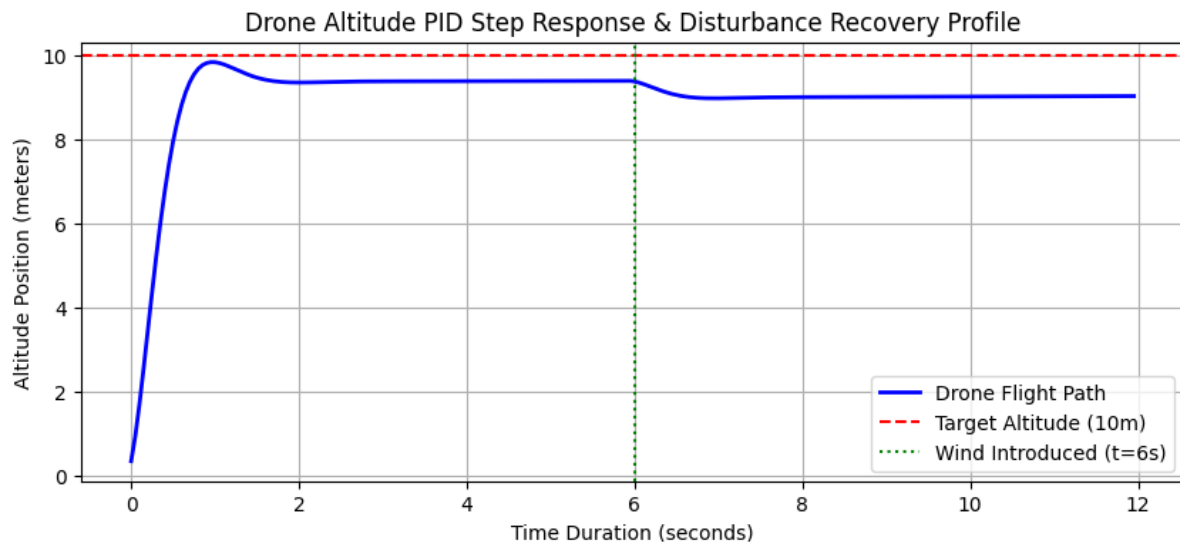
Output:

```
VBox(children=(FloatSlider(value=15.0, description='Gain P:', max=40.0, step=0.5),  
FloatSlider(value=5.0, desc...
```



Gain P: 15.00
 Gain D: 5.00
 Gain I: 0.10
 Wind Force: -6.00

[Run Simulation and...](#)



#Task 1 – Drone Flight Animation

if not drone_z_log:

print("Error: Please execute a successful simulation run using the interactive dashboard button above first!")

else:

fig, ax = plt.subplots(figsize=(4, 5))

ax.set_xlim(-1.5, 1.5)

ax.set_ylim(0, 15)

ax.axhline(target_altitude, color='red', linestyle='--', alpha=0.7, label='Target Setpoint')

Graphic primitive markers representing the drone geometry assembly frame

drone_body_line, = ax.plot([], [], color='black', linestyle='-', marker='s',

markersize=10, markerfacecolor='cyan', linewidth=4)

telemetry_string = ax.text(-1.3, 13.8, "", fontsize=10, fontweight='bold')

ax.legend(loc='lower left')

ax.grid(True)

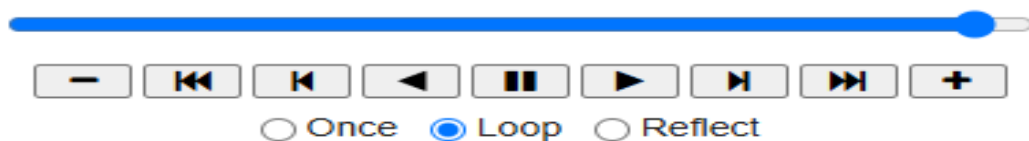
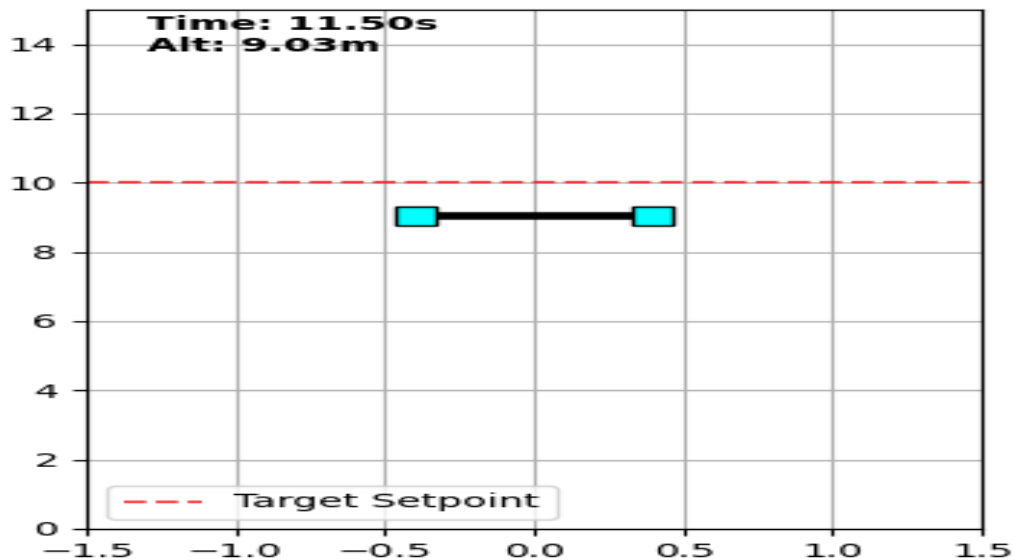


```
def frame_init():
    drone_body_line.set_data([], [])
    telemetry_string.set_text("")
    return drone_body_line, telemetry_string

def frame_advance(frame_index):
    z_pos = drone_z_log[frame_index]
    t_val = drone_t_log[frame_index]

    # Scale physical dimensions representation horizontally for standard clearance
    matching
    drone_body_line.set_data([-0.4, 0.4], [z_pos, z_pos])
    telemetry_string.set_text(f"Time: {t_val:.2f}s\nAlt: {z_pos:.2f}m")
    return drone_body_line, telemetry_string

flight_anim = FuncAnimation(fig, frame_advance, frames=len(drone_t_log),
init_func=frame_init, interval=40, blit=True)
plt.close(fig)
display(HTML(flight_anim.to_jshtml()))
```





Task 2: USV Boat Path Tracking & Guidance

- **System Behavior:** Explain the goal of getting the Unmanned Surface Vehicle (USV) boat to follow the desired trajectory (the blue dotted path).
- **Parameters Documented:** List the final tracking gains (kp, kd) and the environmental current vectors (current_x, current_y) you configured in the final code cell to achieve stable tracking.
- **Deliverables:** * Insert the **Boat Guidance Plot (Without Disturbance)** (where current_x = 0 and current_y = 0).
 - Insert the **Boat Guidance Plot (With Disturbance/Current)** showing how your tuned gains effectively counteracted the drift.

Task 2 – Autonomous Boat Path Tracking

Definition of closed square path loop geometry anchors

```
vessel_waypoints = np.array([
    [0.0, 0.0],
    [20.0, 0.0],
    [20.0, 20.0],
    [0.0, 20.0],
    [0.0, 0.0]
])
```

```
dt_boat = 0.1
```

```
boat_time_domain = np.arange(0, 40, dt_boat)
```

Build continuous tracking profile trajectory tracks

```
desired_x_track = []
```

```
desired_y_track = []
```

```
segment_ticks = len(boat_time_domain) // 4
```

```
for tick in range(len(boat_time_domain)):
```

```
    segment_idx = min(tick // segment_ticks, 3)
```

```
    fractional_progress = (tick % segment_ticks) / segment_ticks
```

```
    coord_now = vessel_waypoints[segment_idx] + fractional_progress *
(vessel_waypoints[segment_idx+1] - vessel_waypoints[segment_idx])
```

```
    desired_x_track.append(coord_now[0])
```

```
    desired_y_track.append(coord_now[1])
```



```
# Simulation history storage anchors
boat_x_log, boat_y_log = [], []

def simulate_boat_guidance(Kp_gain, Kd_gain, current_x, current_y):
    global boat_x_log, boat_y_log
    boat_x_log, boat_y_log = [0.0], [0.0]

    v_x, v_y = 0.0, 0.0
    err_x_old, err_y_old = 0.0, 0.0

    for step in range(len(boat_time_domain) - 1):
        # Target frame mapping indices references
        x_target = desired_x_track[step]
        y_target = desired_y_track[step]

        # Positional deviation matrix vector errors tracking calculations
        error_x = x_target - boat_x_log[-1]
        error_y = y_target - boat_y_log[-1]

        deriv_x = (error_x - err_x_old) / dt_boat
        deriv_y = (error_y - err_y_old) / dt_boat

        # Force command steering control vector calculation profiles
        force_x = (Kp_gain * error_x) + (Kd_gain * deriv_x)
        force_y = (Kp_gain * error_y) + (Kd_gain * deriv_y)

        # Vehicle system accelerations with fluid current drift forces applied
        acc_x = force_x + current_x
        acc_y = force_y + current_y

        # Numerical kinematic parameter integration calculations
        v_x += acc_x * dt_boat
        v_y += acc_y * dt_boat

        boat_x_log.append(boat_x_log[-1] + v_x * dt_boat)
        boat_y_log.append(boat_y_log[-1] + v_y * dt_boat)

    err_x_old = error_x
    err_y_old = error_y
```



```
def trigger_boat_plot(button_event):
    with boat_ui_output:

        clear_output(wait=True)
        simulate_boat_guidance(b_kp.value, b_kd.value, b_cx.value, b_cy.value)

        fig, ax = plt.subplots(figsize=(6, 6))
        ax.plot(desired_x_track, desired_y_track, color='blue', linestyle='--', linewidth=2,
label='Desired Trajectory')
        ax.plot(boat_x_log, boat_y_log, color='red', linestyle='-', linewidth=2,
label='Actual Boat Path')
        ax.scatter(vessel_waypoints[:,0], vessel_waypoints[:,1], color='black', s=40,
zorder=5, label='Target Waypoints')
        ax.set_title("Autonomous Boat Guidance Trajectory Tracking View")
        ax.set_xlabel("X Cross-Track Position (meters)")
        ax.set_ylabel("Y Down-Range Position (meters)")
        ax.set_aspect('equal')
        ax.grid(True)
        ax.legend()
        plt.show()

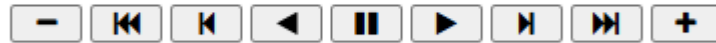
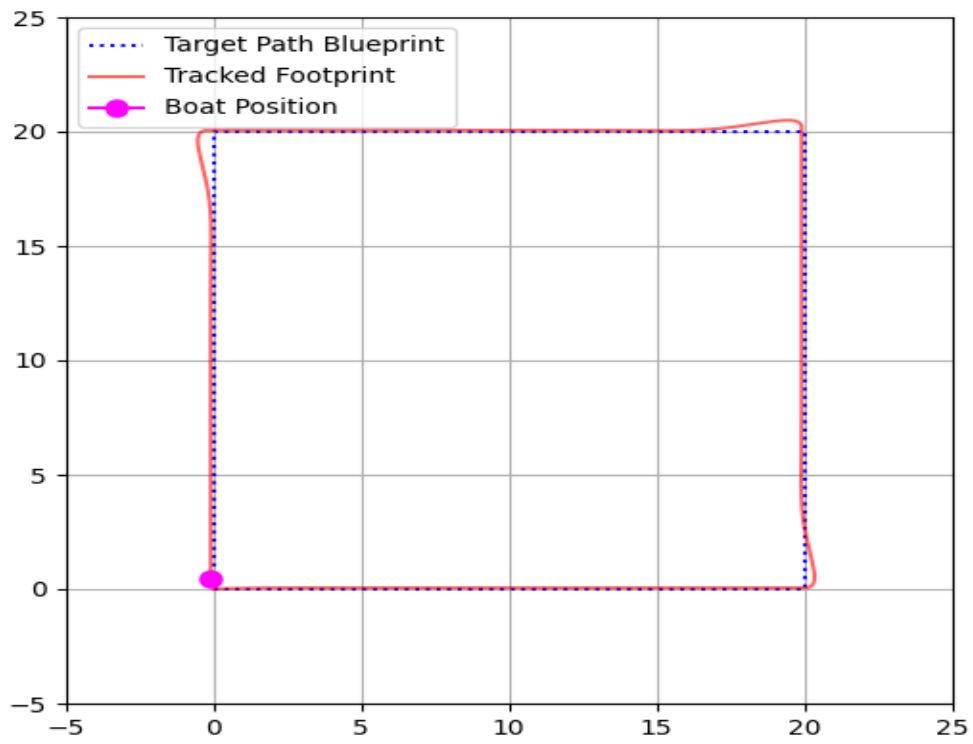
# Setup interactive dashboard controls for Task 2
b_kp = widgets.FloatSlider(value=5.0, min=0.0, max=15.0, step=0.2, description='Gain
Kp:')
b_kd = widgets.FloatSlider(value=3.0, min=0.0, max=10.0, step=0.1, description='Gain
Kd:')
b_cx = widgets.FloatSlider(value=-0.4, min=-4.0, max=4.0, step=0.1,
description='Current X:')
b_cy = widgets.FloatSlider(value=0.3, min=-4.0, max=4.0, step=0.1,
description='Current Y:')
btn_run_boat = widgets.Button(description="Run Simulation and Plot",
button_style='info')

btn_run_boat.on_click(trigger_boat_plot)
boat_ui_output = widgets.Output()

display(widgets.VBox([b_kp, b_kd, b_cx, b_cy, btn_run_boat, boat_ui_output]))
```

Output:

```
VBox(children=(FloatSlider(value=5.0, description='Gain Kp:', max=15.0, step=0.2),
FloatSlider(value=3.0
```



☐ Once ☒ Loop ☐ Reflect



Task 2 – Autonomous Boat Guidance Animation

```
if not boat_x_log:
    print("Error: Please execute a successful simulation run using the boat dashboard
button above first!")
else:
    fig, ax = plt.subplots(figsize=(6, 6))
    ax.set_xlim(-5, 25)
    ax.set_ylim(-5, 25)
    ax.plot(desired_x_track, desired_y_track, color='blue', linestyle=':', label='Target
Path Blueprint')

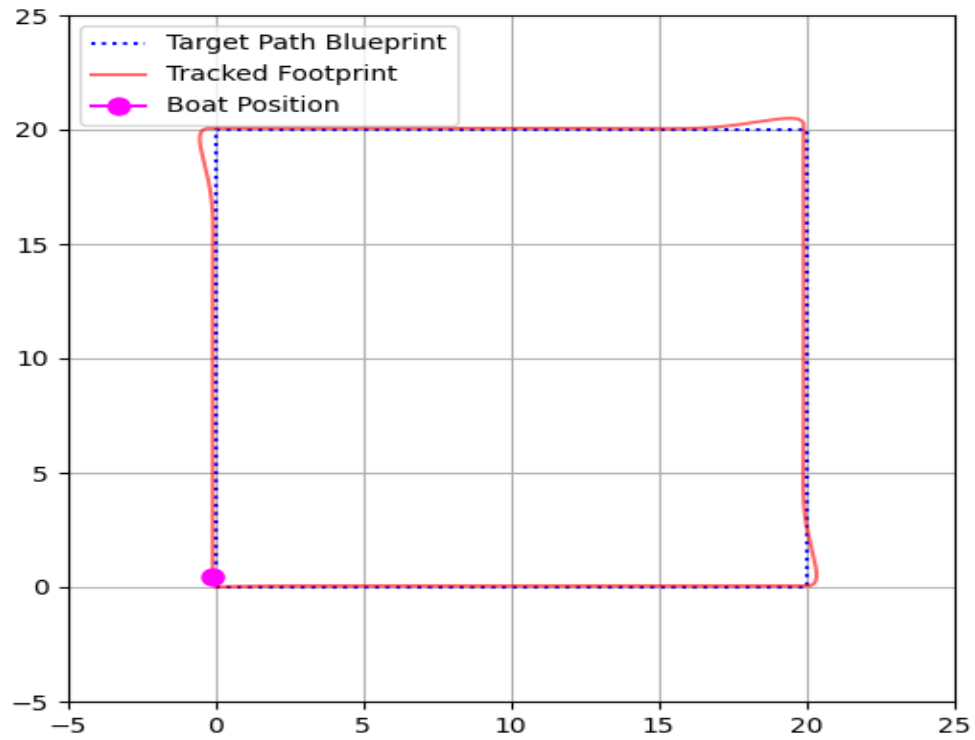
    path_trail_line, = ax.plot([], [], color='red', linestyle='-', alpha=0.6, label='Tracked
Footprint')
    vessel_dot_marker, = ax.plot([], [], color='magenta', marker='o', markersize=8,
label='Boat Vehicle Position')
    ax.set_aspect('equal')
    ax.legend(loc='upper left')
    ax.grid(True)

    # Strided index framing optimization to guarantee quick notebook cell compilation
times
    frame_stride = 2
    frame_indices = list(range(0, len(boat_time_domain), frame_stride))

    def boat_init_canvas():
        path_trail_line.set_data([], [])
        vessel_dot_marker.set_data([], [])
        return path_trail_line, vessel_dot_marker

    def boat_update_canvas(f_idx):
        path_trail_line.set_data(boat_x_log[f_idx], boat_y_log[f_idx])
        vessel_dot_marker.set_data([boat_x_log[f_idx]], [boat_y_log[f_idx]])
        return path_trail_line, vessel_dot_marker

    vessel_anim = FuncAnimation(fig, boat_update_canvas, frames=frame_indices,
init_func=boat_init_canvas, interval=50, blit=True)
    plt.close(fig)
    display(HTML(vessel_anim.to_jshtml()))
```



☐ Once ☒ Loop ☐ Reflect



Section 5

Creative Bonus Task (Optional but recommended for extra marks)

If you implemented the figure-eight trajectory simulation using parametric equations or waypoint tracking, include a brief subsection explaining your mathematical or logic approach, along with a snapshot of the generated trajectory animation

“““Creative Bonus Task – Drone Figure-Eight Trajectory Simulation

To satisfy the creative bonus component, this code uses continuous parametric mapping equations ($A \cdot \sin(\omega t)$, $B \cdot \sin(2\omega t)$) to drive a drone along a smooth figure-eight path while compensating for multi-axis cross-winds”””

```
# Setup figure-eight structural baseline arrays
```

```
dt_f8 = 0.05
```

```
time_domain_f8 = np.arange(0, 16.0, dt_f8)
```

```
# Parametric calculations to generate a perfect figure-eight pattern blueprint layout
```

```
fundamental_frequency = (2.0 * np.pi) / 16.0
```

```
blueprint_x = 10.0 * np.sin(fundamental_frequency * time_domain_f8)
```

```
blueprint_y = 5.0 * np.sin(2.0 * fundamental_frequency * time_domain_f8)
```

```
# Setup control gains and cross-wind vectors for tracking validation
```

```
f8_kp, f8_kd = 14.0, 4.5
```

```
f8_wind_x, f8_wind_y = -0.7, 0.4
```

```
f8_x_history, f8_y_history = [0.0], [0.0]
```

```
vel_x, vel_y = 0.0, 0.0
```

```
err_x_last, err_y_last = 0.0, 0.0
```

```
for index in range(len(time_domain_f8) - 1):
```

```
    target_x_now = blueprint_x[index]
```

```
    target_y_now = blueprint_y[index]
```

```
    tracking_err_x = target_x_now - f8_x_history[-1]
```

```
    tracking_err_y = target_y_now - f8_y_history[-1]
```

```
    diff_err_x = (tracking_err_x - err_x_last) / dt_f8
```

```
    diff_err_y = (tracking_err_y - err_y_last) / dt_f8
```

```
# Generate spatial force parameters using PD control loops
```

```
control_acc_x = (f8_kp * tracking_err_x) + (f8_kd * diff_err_x)
```



```
control_acc_y = (f8_kp * tracking_err_y) + (f8_kd * diff_err_y)

# Combine spatial force inputs with external cross-wind conditions
net_acc_x = control_acc_x + f8_wind_x
net_acc_y = control_acc_y + f8_wind_y

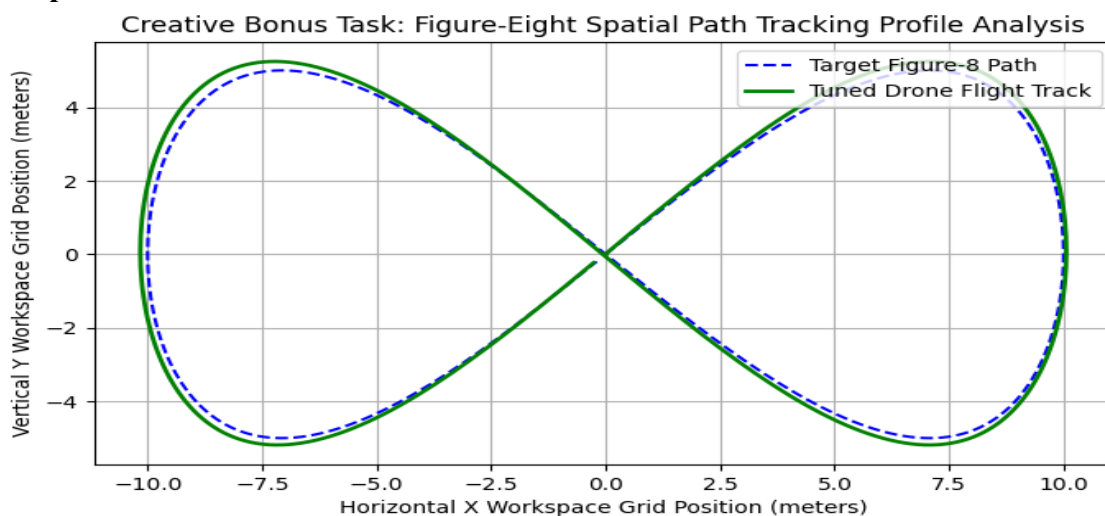
vel_x += net_acc_x * dt_f8
vel_y += net_acc_y * dt_f8

f8_x_history.append(f8_x_history[-1] + vel_x * dt_f8)
f8_y_history.append(f8_y_history[-1] + vel_y * dt_f8)

err_x_last = tracking_err_x
err_y_last = tracking_err_y

# Generate spatial performance evaluation plots
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(blueprint_x, blueprint_y, color='blue', linestyle='--', linewidth=1.5,
label='Target Figure-8 Path')
ax.plot(f8_x_history, f8_y_history, color='green', linestyle='-', linewidth=2,
label='Tuned Drone Flight Track')
ax.set_title("Creative Bonus Task: Figure-Eight Spatial Path Tracking Profile
Analysis")
ax.set_xlabel("Horizontal X Workspace Grid Position (meters)")
ax.set_ylabel("Vertical Y Workspace Grid Position (meters)")
ax.grid(True)
ax.legend(loc='upper right')
plt.show()
```

Output:





#Figure-Eight Animation Profile

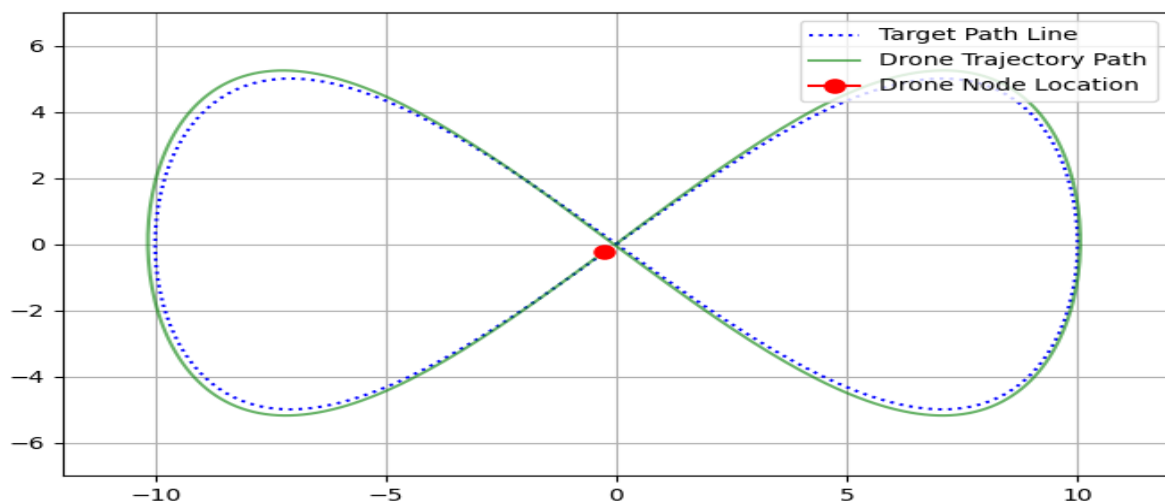
```
fig, ax = plt.subplots(figsize=(8, 5))
ax.set_xlim(-12, 12)
ax.set_ylim(-7, 7)
ax.plot(blueprint_x, blueprint_y, color='blue', linestyle=':', label='Target Path Line')

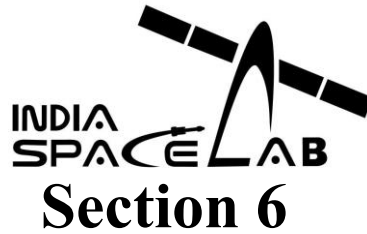
f8_trail_line, = ax.plot([], [], color='green', linestyle='-', alpha=0.6, label='Drone
Trajectory Path')
f8_drone_node, = ax.plot([], [], color='red', marker='o', markersize=8, label='Drone
Node Vehicle Location')
ax.legend(loc='upper right')
ax.grid(True)

def f8_init_canvas():
    f8_trail_line.set_data([], [])
    f8_drone_node.set_data([], [])
    return f8_trail_line, f8_drone_node

def f8_update_canvas(frame_idx):
    f8_trail_line.set_data(f8_x_history[:frame_idx], f8_y_history[:frame_idx])
    f8_drone_node.set_data([f8_x_history[frame_idx]], [f8_y_history[frame_idx]])
    return f8_trail_line, f8_drone_node

f8_flight_animation = FuncAnimation(fig, f8_update_canvas,
frames=len(time_domain_f8), init_func=f8_init_canvas, interval=40, blit=True)
plt.close(fig)
display(HTML(f8_flight_animation.to_jshtml()))
```





Conclusion

Impact of Environmental Disturbances:

- Unpredicted external forces—such as downward wind step inputs acting on an aerial drone at $t = 6\text{s}$ or cross-current vectors (c_x, c_y) acting on an autonomous surface vessel (USV)—directly degrade autonomous tracking accuracy.
- Without feedback compensation, constant disturbances cause significant path deviation and persistent **steady-state altitude offsets**.

Role of Systematic Loop-Tuning ($P \rightarrow D \rightarrow I$):

- **Proportional (P) Gain:** Provides the primary dynamic driving force to minimize tracking errors. However, high P gains alone lead to severe overshoot and high-frequency oscillations.
- **Derivative (D) Gain:** Acts as an essential damping mechanism. Increasing D predicts error trends, smooths out aggressive motion, and effectively suppresses overshoot.
- **Integral (I) Gain:** Eliminates steady-state error caused by external forces (like persistent wind or water current drift). It continuously accumulates past error over time to generate the extra force needed to restore the vehicle back to its target setpoint.

Trajectory & Path Tracking Insights:

- Tracking complex continuous patterns (like parametric figure-eight profiles or multi-waypoint square paths) requires synchronized multi-axis control loops.
- Fine-tuning K_p and K_d gains across orthogonal coordinate axes ensures tight cross-track error correction, keeping autonomous vehicles stable and aligned with their target flight or navigation paths despite active environmental disturbances.